

de control: la ALU, el registro de corrimiento y la lógica de microsecuencia-

miento. Para dar un ejemplo de un código de operación de microinstrucción, consideremos BEGRD, que inicia una lectura en memoria. Usa la función 2 de la ALU (selección de bus A), con lo que $ALUH = 1$ y $ALUL = 0$. También carga el MAR y activa RD. Las demás señales de control quedan inactivas. Ahora consideremos los saltos. Como hemos decidido que sean compatibles con nuestra vieja lógica de microsecuenciamiento, el par MSLH MSLL debe ser 00 para no salto, 01 para salto si N, 10 para salto si Z y 11 para salto incondicional (la compatibilidad ha alcanzado tales proporciones epidémicas, que incluso las máquinas hipotéticas de los libros de texto son compatibles con sus predecesoras), NJUMP genera 01, ZJUMP genera 10 y UJUMP genera 11. Todos los demás códigos de operación generan 00.

Código de operación de microinstrucción		Líneas de control											
		ALUL		SHL		AMUX		MAR		RD		MSLH	
		ALUH	SHH	NZ	AND	MBR	WR	MSLL					
0	ADD			+	+								
1	AND	+		+	+								
2	MOVE	+		+	+								
3	COMPL	+	+	+	+								
4	LSHIFT	+	+	+	+								
5	RSHIFT	+		+	+	+							
6	GETMBR	+		+	+	+							
7	TEST	+		+									
8	BEGRD	+						+		+			
9	BEGWR	+						+	+		+		
10	CONRD	+								+			
11	CONWR	+									+		
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

Fig. 4-19. Señales de control para cada código de operación de microinstrucción. Un signo + significa que la señal está activa; un blanco significa que no lo está.

Ahora viene la parte interesante. ¿Cómo construiremos un circuito de cuatro entradas (los bits de código de operación) y trece salidas (las señales de control) que calcule la función de la figura 4-19? Pues con el uso de una o varias PLA (o ROM). La figura 4-19 es una forma algo peculiar de representar 13 tablas de verdad, una por columna, de cuatro variables cada una, donde el número de fila define implícitamente el valor de las cuatro variables. Entonces la cuestión de cómo construir el circuito se reduce a cómo implementar una tabla de verdad. La mejor forma de hacerlo consiste en emplear una PLA de cuatro entradas y 13 salidas. Si no la podemos conseguir, podemos usar tres 74S330, que son PLA de 12 entradas y 6 salidas. Si denotamos los cuatro bits de código de operación con las letras *A* a *D*, del más significativo al menos significativo, algunas de las salidas son:

$$ALUL = \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD = \bar{A}\bar{B}D$$

$$SHH = \bar{A}B\bar{C}\bar{D}$$

$$MAR = A\bar{B}\bar{C}\bar{D} + A\bar{B}CD = \bar{A}\bar{B}\bar{C}$$

$$MSLH = ABC\bar{D} + ABCD = ABC$$

Sólo deben generarse internamente 15 términos producto, ya que $A\bar{B}\bar{C}D$ no se necesita.

Habiendo rediseñado el hardware, necesitamos reescribir el microprograma. Lo muestra la figura 4-20. Se han dejado las mismas etiquetas para que sea más fácil comparar los dos microprogramas. También se ha conservado la sintaxis. Podíamos haberlo escrito usando un lenguaje ensamblador típico (por ejemplo, con los códigos de operación de la figura 4-17), pero hemos usado de nuevo LMAN porque es mucho más fácil de leer. Nótese que las sentencias LMAN de la forma *alu = reg* usan la microinstrucción TEST para ajustar los bits *N* y *Z*. Asegúrese el lector de que entiende la diferencia entre el microprograma en binario, tal como se carga en la memoria de control, y la versión en lenguaje ensamblador que se da en el texto.

Aunque más largo, este microprograma es más simple que el anterior puesto que cada línea sólo realiza una operación. En consecuencia, muchas líneas del original han tenido que dividirse en dos, tres, e incluso cuatro líneas en éste. Otra causa que aumenta el número de microinstrucciones es la ausencia de microinstrucciones de tres direcciones. Véase, por ejemplo, las líneas 22 y 27 del primer microprograma.

El microprograma original usa 79 palabras de 32 bits, que dan un total de 2528 bits de memoria de control. El segundo usa 160 palabras de 12 bits, que hacen un total de 1920 bits. La diferencia representa un ahorro de 24% de memoria de control. En una computadora integrada en una pastilla, también


```

0:  mar := pc; rd;
1:  rd;
   pc := pc + 1;
2:  ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3:  tir := lshift(tir);
   if n then goto 19;
4:  tir := lshift(tir);
   if n then goto 11;
5:  alu := tir;
   if n then goto 09;

6:  mar := ir; rd; {LODD}
7:  rd;
8:  ac := mbr;
   goto 0;

9:  mar := ir; mbr := ac; wr; {STOD}
10: wr;
    goto 0;

11: alu := tir;
    if n then goto 15;

12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;

15: mar := ir; rd; {SUBD}
16: rd;
17: ac := ac + 1;
18: a := mbr;
    a := inv(a);
19: ac := ac + a;
    goto 0;

20: tir := lshift(tir);
    if n then goto 25;
21: alu := tir;
    if n then goto 23;
22: pc := ir;
    pc := band(pc, amask);
    goto 0;

23: alu := ac; {JZER}
    if z then goto 22;
24: goto 0;

25: alu := tir;
    if n then goto 27;
26: pc := ir; {JUMP}
    pc := band(pc, amask);
    goto 0;

27: ac := ir; {LOCO}
    ac := band(ac, amask);
    goto 0;

28: tir := lshift(tir);
    if n then goto 40;
29: tir := lshift(tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;

31: a := ir; {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0;

33: a := ir; {STOL}
    a := a + sp;
34: mar := a; mbr := ac; wr;
    wr;
    goto 0;

35: alu := tir;
    if n then goto 38;

36: a := ir; {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;

```

Fig. 4-20. El microprograma del Mic-2.


```

38: a := ir; {SUBL}
   a := a + sp;
39: mar := a; rd;
   rd;
   goto 99;
40: tir := lshift(tir);
   if n then goto 46;
41: alu := tir;
   if n then goto 44;
42: alu := ac; {JNEG}
   if n then goto 22;
43: goto 0;
44: alu := ac; {JNZE}
   if z then goto 0;
45: pc := ir;
   pc := band(pc, amask);
   goto 0;
46: tir := lshift(tir);
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr := pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift(tir);
   if n then goto 65;
51: tir := lshift(tir);
   if n then goto 59;
52: alu := tir;
   if n then goto 56;
53: mar := ac; rd; {PSHL}
   rd;
54: sp := sp + (-1);
55: a := mbr;
   mar := sp; mbr := a; wr;
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
59: alu := tir;
   if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar := sp; mbr := ac; wr;
   wr;
   goto 0;
62: mar := sp; rd; {POP}
63: rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift(tir);
   if n then goto 73;
66: alu := tir;
   if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp := a;
   goto 0;
73: alu := tir;
   if n then goto 76;
74: a := ir; {INSP}
   a := band(a, smask);
75: sp := sp + a;
   goto 0;
76: a := ir; {DESP}
   a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
   sp := sp + a;
   goto 0;

```

Fig. 4-20. (continuación).

FACTA 124, 125

representa un ahorro de 24% en el espacio de la pastilla (chip) ocupado por la memoria de control, lo que lo hace más barato y fácil de fabricar. El precio que hay que pagar por la memoria de control más pequeña es que han de ejecutarse más microinstrucciones por macroinstrucción. Normalmente, esto hace más lenta la máquina. En consecuencia, las máquinas veloces y caras tienden a ser horizontales y las más lentas y baratas, verticales.

La existencia de microinstrucciones muy codificadas, como las del Mic-2, plantea serios problemas conceptuales acerca de lo que es la microprogramación. El juego de microinstrucciones de la figura 4-17 podría pasar casi por el juego de instrucciones del **lenguaje de máquina convencional** de una micro o minicomputadora muy simple. La PDP-8, por ejemplo, es una minicomputadora con palabras de 12 bits cuyas instrucciones no son mucho más potentes que éstas. Considerando que el "significado" de las instrucciones está determinado por el hardware (la PLA o circuito del decodificador de OP), se podría decir que el Mic-2 es realmente una máquina no microprogramada en la que se ejecuta un intérprete para otra máquina. Si el microprograma de la máquina vertical se guardara en la memoria principal (como sucede en la IBM 370/145, por ejemplo), la distinción entre una máquina microprogramada muy vertical y una cableada (no microprogramada) perdería aún más nitidez. Véase a Dasgupta (1979) si se desea profundizar en los aspectos de codificación y paralelismo en el microcódigo.

4.5.2. Nanoprogramación

Los diseños vistos hasta ahora tenían dos memorias: la central (utilizada para guardar el programa de nivel 2) y la de control (utilizada para guardar el microprograma). Con una tercera memoria, la **nanomemoria**, se pueden realizar interesantes compromisos entre la organización horizontal y la vertical. La nanoprogramación es apropiada cuando se repiten varias veces muchas microinstrucciones. El microprograma de la figura 4-16 no tiene esa propiedad. La microinstrucción más frecuente es la que sólo tiene *lec* y únicamente aparece cinco veces.

La figura 4-21 ilustra el concepto de nanoprogramación. La parte (a) muestra un microprograma de n microinstrucciones de w bits cada una. Se necesitan nw bits de memoria de control en total para guardarlo. Supongamos que un estudio detallado del microprograma muestra que sólo se usan m microinstrucciones diferentes de las 2^w posibles, siendo $m \ll n$. Se podría utilizar una nanomemoria especial de m palabras de w bits para guardar cada microinstrucción única. En el microprograma original se podría reemplazar cada instrucción por su dirección en la nanomemoria. Como la nanomemoria sólo contiene m palabras, la memoria de control necesaria tiene solamente $\log_2 m$ (redondeado al entero mayor) bits de anchura, como se muestra en la figura 4-21(b).